Cross-Functional Microservices Project Assignment

Main Assignment Brief

Module	Software Architecture and Process Models
Level	Level 2
Assignment Duration	7 weeks
Document	01 of 07
Cohort Structure	56 students - 8 teams of 7 students
Project Structure	2 projects - 4 teams collaborate on each project

1. Assignment Overview

This assignment provides practical experience across the full software project lifecycle: project initiation, requirements analysis, architecture, iterative development, cross-functional teamwork, cross-team integration, testing, deployment, and project closure. Students will work in eight teams of seven. Four teams will collaboratively deliver an Apartment Management System, while the remaining four teams will deliver a University Services Management Platform.

Core expectation

Each team owns a clearly bounded business domain and its microservices, while all four teams assigned to the same project remain jointly responsible for the integrated product. A team cannot obtain full marks by delivering an isolated service that does not integrate with the shared application.

2. Learning Objectives

- Apply project initiation, scope definition, risk management, and iterative planning practices.
- Translate a case scenario into functional requirements, non-functional requirements, user stories, and acceptance criteria.
- Design a secure microservice architecture with clear service boundaries, databases, APIs, and deployment components.
- Develop a modular React and TypeScript frontend connected to Spring Boot REST APIs.
- Use Jira to manage work and GitHub to demonstrate collaborative development, review, integration, and individual contribution.
- Plan and execute unit, API, integration, system, security, and user acceptance testing appropriate to a functional prototype.
- Collaborate across functional roles and across four independent teams to release one coherent system.
- Evaluate the project through demonstrations, evidence-based reporting, retrospectives, and lessons learned.

3. Team and Project Allocation

Project	Assigned Groups	Project Goal
Project A - Apartment Management System	Groups 1, 2, 3, and 4	Deliver an integrated platform for apartment administration, residents, property and occupancy, billing, and day-to-day operations.
Project B - University Services Management Platform	Groups 5, 6, 7, and 8	Deliver an integrated platform for students and staff to access university facilities, reservations, service requests, events, and university communications.

4. Team Roles and Cross-Functional Working

Each eight-member team must allocate primary responsibilities. Students may hold more than one role and every student must contribute to technical or quality deliverables, not only coordination documentation.

Suggested Primary Responsibility	Expected Contribution
Scrum Master	Facilitates planning, ceremonies, impediment removal, reporting, and evidence collection.
BA	Maintains domain scope, user stories, acceptance criteria, traceability, and change control.
Team Lead	Coordinates API contracts, architecture decisions, shared frontend integration, and weekly cross-team synchronization.
UI/UX and Frontend Developer	Maintains UX consistency, shared design conventions, frontend module structure, accessibility, and reviews.

Suggested Primary Responsibility	Expected Contribution
Frontend Developer	Implements assigned feature modules, state management, API integration, validation, and tests.
Backend Developer 1	Implements domain services, security, validation, persistence, and automated tests.
Backend Developer 2	Implements complementary services, integrations, database scripts, API documentation, and tests.
QA / DevOps	Coordinates test planning, defects, CI checks, Docker configuration, deployment, and release evidence.

Teams may adapt the role names, but the Team Lead is mandatory. Role allocation must be recorded in the project proposal and updated when responsibilities change.

5. Required Technology Stack

Area	Required Standard
Frontend	One shared React application using TypeScript and Redux. Teams own feature modules within the shared codebase. A consistent component library and design system must be used.
Backend	Spring Boot microservices exposing REST APIs. Code should follow a clear controller-service-repository/domain organization and use Hibernate/JPA.
Database	MySQL. Each microservice must own a separate database or schema and may not directly read another service database.
Security	JWT-based authentication, role-based authorization, secure password storage, request validation, safe error responses, and protected endpoints.
API Documentation and Testing	OpenAPI/Swagger documentation and Postman collections.
Containerization	Docker images for services and Docker Compose for local integrated execution.
Version Control	One GitHub organization per overall project; one repository per microservice; one shared frontend repository.
Project Tracking	One Jira project per eight-member team. Each team maintains its own backlog and synchronizes dependencies with the other three teams.
CI/CD	GitHub Actions must at minimum build and test code on pull requests. Automated deployment is encouraged but not required.
Deployment	A working integrated release must be deployed to an instructor-approved cloud or university environment.

6. Mandatory Microservice Architecture Requirements

- Business capabilities must be separated into independently deployable microservices with explicit ownership.
- An API Gateway must provide a controlled entry point for the frontend and route requests to backend services.
- A centralized authentication service must issue JWTs; services must validate authorization for protected operations.
- Each microservice must own its data store or schema. Cross-service database access is prohibited.
- Inter-service communication must use documented REST APIs. Teams must define timeout, error, and failure behavior.
- All APIs must use consistent response conventions, validation rules, HTTP status codes, and centralized exception handling.
- Docker Compose must run the complete local system, including services and databases needed for integration testing.
- Services must expose basic health information and produce usable logs that allow failures to be diagnosed.

- Swagger/OpenAPI specifications and versioned API contracts must be maintained throughout development.
- The integrated frontend must use modular feature ownership while sharing navigation, authentication state, and design conventions.

Optional advanced features

RabbitMQ or Kafka, service discovery, distributed tracing, Kubernetes, centralized log aggregation, circuit breakers, and automated cloud deployment may be implemented for additional technical merit, but they must not replace completion of the mandatory scope.

7. Prototype Scope Standard

The required outcome is a functional prototype, not a commercial production system. The prototype must nevertheless demonstrate secure design, meaningful business rules, data persistence, integrated workflows, error handling, testing, and deployment.

Minimum Prototype Capability	Evidence Expected
Authentication and authorization	Login, JWT use, role restrictions, and protected API evidence.
Core business workflows	At least two end-to-end workflows that cross team or service boundaries.
Data operations	Validated create, read, update, and relevant status-transition operations for core entities.
Search and usability	Appropriate filtering, search, pagination, clear navigation, validation messages, and responsive layouts.
Integration	Shared frontend, API Gateway, documented API contracts, and cross-team dependency handling.
Quality	Automated tests, Postman collection, test execution evidence, defect tracking, security checks, and simulated UAT.
Deployment	Accessible integrated release, deployment guide, release notes, and reproducible configuration.

9. Cross-Team Governance

- Each project establishes a Project Integration Council consisting of the four Team Leads.
- The council conducts a weekly synchronization meeting and records decisions, dependencies, interface changes, integration risks, and action owners.
- API contract changes must be communicated before implementation, documented in the project documentation repository, and reflected in dependent Jira issues.
- Shared frontend changes must follow pull-request review and avoid breaking feature modules owned by other teams.
- No team may silently change shared authentication, gateway routes, common data definitions, or response formats.
- Integration responsibility is shared by all four teams. The Team Lead coordinates integration but does not become solely responsible for it.

Teams may select their own sprint duration. The selected sprint must be justified, time-boxed, visible in Jira, and include at least two completed development iterations before the final release and closure activities.

10. Required Deliverables

Category	Team-Level Deliverables	Project-Level Shared Deliverables
Project management	Team plan, backlog, sprint backlogs, user stories, progress reports, risk log, change log, sprint reviews, retrospectives.	Project proposal/charter, integration plan, shared milestone plan, dependency log, project closure report.
Requirements	Domain SRS sections, functional and non-functional requirements, use cases/user flows, business rules, traceability.	Consolidated scope, cross-domain workflows, shared glossary, system-wide non-functional requirements.
Design and architecture	Service design, ERD/data model, API specification, UI module designs, Architectural Decision Records (ADRs), threat considerations.	System context, microservice architecture, deployment architecture, security model, API contract registry, shared UX conventions.
Software	Owned microservices, database scripts, frontend modules, tests, configuration examples, Docker files.	Integrated web application, API Gateway, authentication integration, Docker Compose, deployment scripts, tagged release.
Quality assurance	Test cases, unit/API test evidence, defect records, service integration results.	Test strategy, end-to-end test execution report, security review, performance observations, simulated UAT report.
Documentation and release	Service README, setup instructions, technical notes, contribution evidence.	Swagger index, Postman collection, user guide, administrator guide, deployment guide, release notes, dependency list.
Academic submission package	Individual contribution statement and team evidence.	Complete repository list, database scripts, configuration templates without secrets, final presentation, demonstration, closure and lessons learned.

11. Assessment Structure

Assessment Area	Weight	Primary Evidence
Project initiation, planning, cross-team coordination, and Jira usage	15%	Proposal, plan, backlog quality, sprint evidence, risks, dependency coordination, and progress visibility.
Requirements analysis	15%	SRS, user stories, acceptance criteria, use cases, business rules, and traceability.
Architecture and technical design	20%	Service boundaries, data ownership, API contracts, security, deployment design, UX architecture, and Architectural Decision Records (ADRs).
Software implementation and integration	25%	Working features, code quality, secure APIs, frontend modules, cross-service workflows, GitHub collaboration, and integrated release.
Testing and quality assurance	15%	Test plan, automated tests, Postman tests, integration/system tests, defects, security, performance observations, and UAT simulation.
Deployment, demonstration, and documentation	5%	Deployed prototype, reproducible setup, guides, release notes, presentation, and live demonstration.
Individual contribution	5%	GitHub commits and pull requests, code reviews, Jira work, authored deliverables, and documented contribution.

12. Final Evaluation Activities

- Project presentation explaining the problem, scope, process, architecture, and outcomes.
- Live demonstration of the integrated deployed prototype.
- Architecture explanation covering service boundaries, security, databases, APIs, and deployment.
- Jira demonstration showing planning, progress, bugs, changes, dependencies, and sprint evidence.
- GitHub contribution and pull-request review.
- Testing evidence demonstration, including failed and corrected cases where available.
- Cross-team retrospective and lessons learned.

13. Submission Principles

- All repositories and Jira projects must remain accessible to the teaching team until marking is completed.
- Secrets, real passwords, personal data, and access tokens must never be committed to GitHub.
- All borrowed code, libraries, templates, and generated content must be acknowledged.
- Evidence created after the final deadline may not be considered unless an authorized extension applies.
- A student whose contribution cannot be verified may receive a reduced individual contribution mark even when the team product is strong.
- Academic integrity requirements apply to source code, documentation, designs, test evidence, and reports.